

Özetle,

Tek doğru yok.

Analiz safhasında yeni gereksinimler doğabilir.

Açık olmayan noktalar ve hatalar olabilir.

Müşteriyle tekrar tekrar temasa geçmek zorunda kalabiliriz.

OCL ile sıkı düşünüp bu tip noktaları minimuma indirebiliriz.

Davranış Modellemesi

Davranış Modellemesi

Bu ana kadar incelediğimiz yapısal modeller, sistemin her zaman doğru olan özelliklerini tanımladı.

Sistem dış uyarılara nasıl tepki verecek?

- UML'de sistem davranışlarını göstermek için de diyagramlar bulunur.
- Sistem özelliklerini kesin tanımlamak için durum diyagramlarına da bakacağız.

Durumlar (States)

Durum: Verilen bir anda sistem değerleri kümesinin soyut bir tanımı.

Örnek: Dışarıda yağmur yağıyor.

- Zamanda seçilen bir noktada havadaki yağış miktarı için bir tahmin ya da soyutlama.

Daha karışık örnek: Dik kesişen 10 sokak ve 8 caddesi olan hayali bir kasabamız olsun. İçinde tek bir araba olan kasabanın durumu nedir?

- Örneğin, arabamız 3.sokak ve 5. caddede olsun.
- Bunun gibi kaç olası durum vardır? $10 \times 8 = 80$
- Kasabada iki araba olursa? $80 \times 80 = 6400$

Bir sistemin durum uzayı, göz önüne aldığımız değişken sayısı (ve değişkenlerin alabileceği değerlere) katında artar.

- Buna durum patlaması (state explosion) problemi adı verilir.

Olaylar (Events)

Tekil, ani, fark edilebilir bir oluşumdur. Sistem tepki vereceği için, bir uyarı olarak da düşünülebilir.

Sistemdeki olaylar senkron ya da asenkron olabilir.

- **Asenkron:** Rasgele oluşurlar. Ani yığılma da olabilir, zamana da yayılabilir.
- **Senkron:** Periyodik aralıklarla olanlar. Sistemde yoğunlukla bu olayları kontrol eden bir saat ya da olay döngüsü bulunur.

Olaylar, sistemde bir durum değişikliğine neden olurlar. Buna **durum geçişi** (state transition) adı verilir.

- Örneğin, ÖSYM sonuç mektubunu aldığınızda, hayatınızda önemli bir değişiklik olur.

Olaylar kullanıcı eylemlerinin sonucu olarak doğabilir (ör. Ekranda bir düğmeye basmak).

Verideki değişiklikler ile oluşabilir (ör. Sıcaklığın 100 dereceyi geçmesi)

Olaylar zaman geçtikçe biriktirilebilir.

UML Olay Tasnifi

UML'de değişik olay tipleri mevcuttur.

Sinyaller (Signal): Asenkron bildirimler

Metot çağrılarları (Method call): Senkron işlem yürütme/başlatma

Veri durumları (Data conditions): Veride durum değişimi .

Zaman akışı

Durum mu Olay mı?

Jesse Owens'ın Berlin'de 1936'da 100 metre olimpiyat şampiyonu olması

- Durum

Mor renk

- Hiçbiri. Sadece bir özelliğin değeri

Binanın yangın söndürme sisteminin bir yangın nedeniyle devreye girmesi

- Olay

Telefonun çalması

- Olay

Yağmurlu bir gün

- Durum

Saatın zilinin 12'yi vurması

- Olay

Uluslararası Uzay İstasyonu

- Hiçbiri. Sadece çok karmaşık bir nesne

Bilgisayarınızın ekran koruyucusu açılıyor.

- Olay

Modelleme Teknikleri

Olaylara tepki veren sistemlere reaktif sistem adı verilir.

Geleneksel sistemlerde işlerin sırasını kontrol etme sorumluluğu sistemdedir.

- Burada ise sistemde birşeylerin olmasına yol açan uyarılar dış dünyadan gelir.

İlk olarak en basiti olan kombinatoryal sistemlere,

- Sadece durumlar, olay yok

Sonra sıralı (sequential) sistemlere,

- Her durum doğrusal olarak bir diğerinden sonra olaylarla sıralıdır.

En son da en karmaşık olan eşzamanlı (concurrent) sistemlere bakacağız.

- Asankron eşzamanlı sistemlerde pek çok durum ve olay vardır, olaylar zamanda kestirilemeyen anlarda oluşabilir.

Kombinatoryal Modelleme

Yalnızca basit kombinatoryal sistemlerin mantığı tanımlanır.

Sonraki durumlar sadece girdiler tarafından belirlenir.

Önceki durumların geçmişi, sonraki durumu etkilemez.

İki denk şekil: Karar ağacı ve karar tablosu

Karar Tablosu

Sistemin davranışını etkileyebilecek pek çok durum olduğunda kullanılır.

Girdi koşulları ve bu girdilere göre çıktı tepkileri halinde tanımlanır.

- Girdi kombinasyonları, sonucu oluşturma şeklini belirler. (Kombinatoryal)

Tablonun sütunlarında girdiler ve çıktılar yer alır.

Her satırda değişik bir girdi kombinasyonu yer alır.

Ör: Bir atölyemiz ve üç şalterimiz olsun. İki çıktıyı kontrol etsinler.

Girdi			Çıktı	
Ana şalter	Işık şalteri	Güç şalteri	Işık	Motor
+	+	+	+	+
+	+	-	+	-
+	-	+	-	+
+	-	-	-	-
-	+	+	-	-
-	+	-	-	-
-	-	+	-	-
-	-	-	-	-

Karar Ağacı

Karar tablosunun grafik gösterimi

Kararlar sıralı olarak alınıp çıktı ağaçta bir yol olarak izlenebilir.

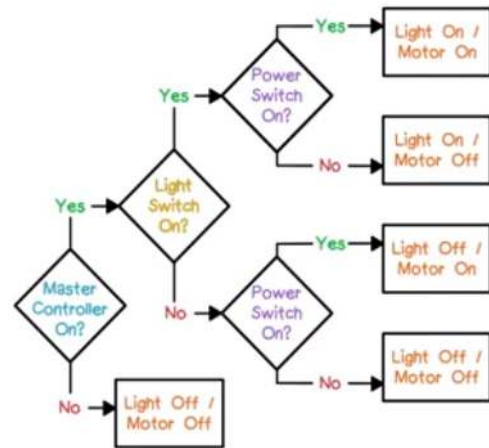
Tamamen aynı bilgi, farklı gösterim.

Eşkenar dörtgenler kararları, dikdörtgenler eylemleri gösterir.

Diyagramdaki bağlantılar olumlu ya da olumsuz karar cevabını gösterir.

Bazı düğümler tekrar edebilir (karar tablosunda da benzer durum vardı)

Girdi olasılıkları arttıkça, ağaç yönetilemez bir hale gelir.



Sıralı (Sequential) Sistemler

Kombinatoryal'den farklı olarak sıralı ve eşzamanlı sistemlerde önceden olanların tutulduğu bir tarihçe/hafıza vardır.

Önceden bir durumdaydık, bu duruma bağlı olarak olaylar yeni bir duruma bizi taşıdı.

Bu gibi sistemlere sonlu durum sistemleri de denir. Çünkü bulunabilecekleri durumları belirli bir sayıya kısıtlarız.

Sonlu durum makineleri ile modelleme yapabiliriz.

Durum Geçiş Tablosu

FSM'leri farklı şekillerde gösterebiliriz. Tablo şekli ile başlayalım.

Satırlarda durumlarımız var.

- 1. sütun durumun adı
- 2.sütun geçişe sebep olan girdi olayı,
- 3.sütun geçiş ile alınacak çıktı aksiyonu
- 4.sütun sonraki durum.

Belirli bir durumda olan sistem için, belirli bir uyarı verildiğinde (oluştığında), oluşacak tepki ve sistemin bulunacağı (yeni) durum tanımlanır.

Durum Geçiş Tablosu

Garaj kapısı örneği: Motor kapıyı yukarı itebilir, aşağı çekebilir, durdurabilir. Motorun çalışıp durmasını sağlayan bir düğme olsun. Kapı tamamen açılabilir, tamamen kapanabilir, ya da arada bir yerde durdurulabilir.

- Düğmeye bastığınızda olacaklar, mevcut durumunuza göre değişecektir.
- Durmuşsa ve bastıysak, aksi yönde hareket edecektir. Yukarı çıkıyorsa ve bastıysak, kapı duracaktır.
- İniyorsa ve bastıysak, sadece durmayıp, güvenlik için açılacaktır.
- Sensörler ile tam açık/tam kapalı durumunu anlayabiliriz.

Kaç durum vardır?

- Kapı açık, motor kapalı
- Kapı kapalı, motor kapalı
- Kapı yarım açık, motor kapalı
- Kapı aşağı, motor açık
- Kapı yukarı, motor açık
- Kapı yarım açık, motor (geçici) kapalı

Kaç olay vardır?

- Düğme
- Sensör açık
- Sensör kapalı

Garaj Kapısı Durum Geçiş Tablosu

Mevcut Durum	Girdi	Eylem	Sonraki Durum
Kapı kapalı motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor
Motor yukarı çalışıyor	Kapı açıldı sensörü	Motor durdur	Kapı açık motor duruyor
Motor yukarı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor
Kapı açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor
Motor aşağı çalışıyor	Kapı kapandı sensörü	Motor durdur	Kapı kapalı motor duruyor
Motor aşağı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor

Durum ve olay sayısı arttıkça tablo anlaşılmas bir hal alır.

Garaj Kapısı Durum Geçiş Diyagramı

Sonlu Durum Makinesinin grafik gösterimi

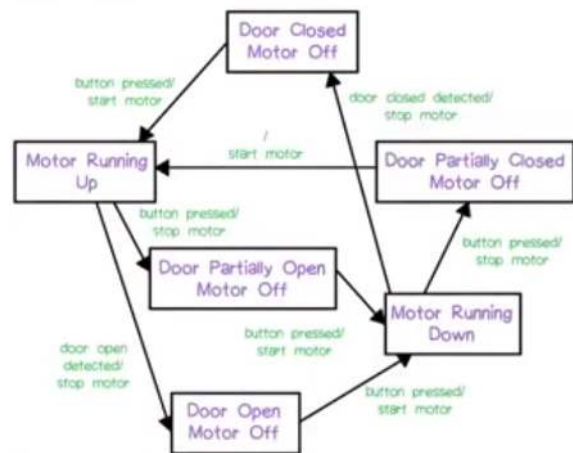
Düğümler oval ya da dikdörtgen ile gösterilir

Bağlantılar durum geçişleridir. Eylem ve genellikle **geçiş** ile isimlendirilebilir.

Düğümlerin yerlerinin anlamsal bir manası yoktur.

Zamanda bir anda, belirli bir durumda olduğunuzu, bir olay olana dek o durumda kalacağınızı, olaya göre uygun geçişi yapacağınızı düşünerek hazırlayabilirsiniz.

Dikkat! Kapı yarım kapalı-motor duruyor iken, güvenlik nedeniyle eylemsiz olarak motor yukarı durumuna geçiş yapıyoruz. (Epsilon geçiş)

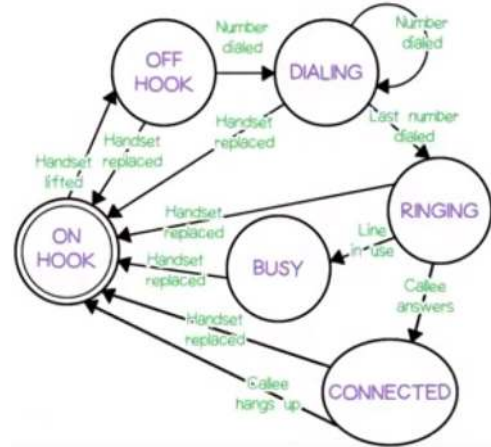


Telefon örneği

Durumlar:

- Telefon kapalı (Çift çizgili, sistemin başlangıç durumu)
- Telefon açık
- Tuşlanıyor (Kendine geçişli, 1. 2. ... Numara çevriliyor. Her aşama ayrı durum)
- Çalıyor
- Meşgul
- Bağlı

Diyagramda gereksiz etiketli geçişler var. Eşzamanlı sistemleri modellerken basitleştirerek iyileştireceğiz.



Durum Geçiş Diyagramı Problemleri

Çok fazla geçiş (ok) var

N durum ve M olay var ise, $N \times M$ olası geçişiniz var.

Çok fazla durum var.

Olabilecek şeylere göre, durumlar da üstel olarak artıyor.

İç içe bulunma çözümü yok.

Çevrilecek telefon numaraları kaç tane olmalı? Kaç kere dönecek?

Durum Diyagramı (Statechart)

Bahsettiğimiz problemleri kısmen çözmek üzere durum diyagramlarını kullanırız.

- Her zaman bu diyagramın da yeterli gelemeyeceği karmaşıklıkta sistemlerle karşılaşabiliriz.

David Harel tarafından geliştirilmiştir.

- Durum geçiş diyagramlarına yaptığı iyileştirmelere durum diyagramı adını vermiştir.
- Karmaşıklığı yönetebilmek için çeşitli mekanizmalar geliştirmiştir.

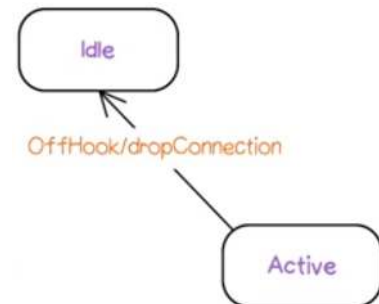
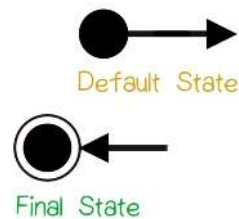
Durum diyagramları UML'in bir parçasıdır ve araçlar tarafından desteklenir.

Durum Diyagramı Sembolleri

Oval ya da dikdörtgen yerine kenarları yuvarlatılmış dörtgen kullanır. İçinde durumun adı yazılır.

Durumlar oklar ile birbirine bağlanır. Olay / aksiyon

Başlangıç / varsayılan durum içi dolu büyük nokta, son durum da çerçeveli hali ile belirtilir.

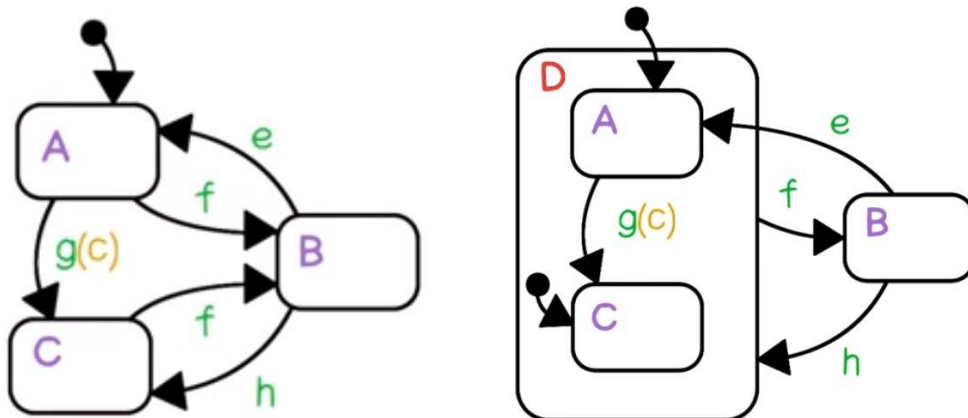


Durum Diyagramı Eklentileri

Durum diyagramları ile yeni özellikler de eklenir.

1. Derinlik (iç içe gruplama): Belirli bir durumun kendi durum makinesi de olabilir. Oraya züm yapabiliriz.
2. Eşzamanlılık (Concurrency): Aynı anda devam eden iki süreç var, bunlarda farklı sayıda durum olabilir. Durum geçiş diyagramlarında olasılıkların çarpımı kadar durum sayısı ile halledebiliyorduk. Burada ayrı ayrı ele alabiliyoruz.
 - Senkronizasyonu da yayım (broadcast) olayları ile halledebiliyoruz.
3. Koşullu Geçişler
4. Giriş/Çıkış eylemleri / aktiviteleri
5. Olay parametreleri
6. Durum geçmişi
7. Varsayılan durumlar

Durum Diyagramı İç içe Gruplama

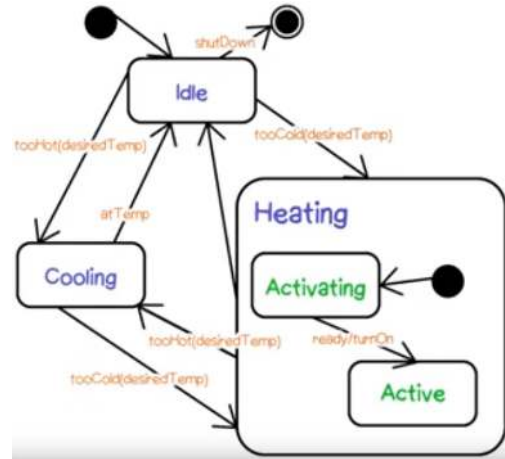


İç içe Gruplama Örneği

Örnek: İklimlendirme Aleti

Isıtma, iki durumlu bir alt sistem

- Varsayılan durumu "activating"
- İç durumların ikisinden de çıkabilecek ikişer ok yerine, gruptan iki ok çıkıyor.



İç içe Gruplama Örneği

Örnek: Printer Sistemi

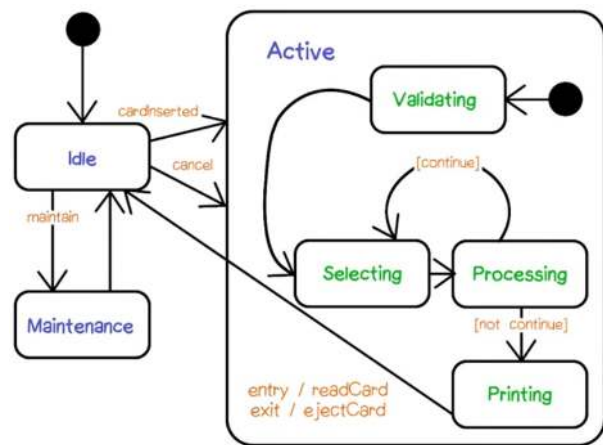
Üç tane üst durum var, "idle", "maintenance", "active"

Active içinde 4 durum var. Idle'dan gelen geçişler, varsayılan "validating"e bağlanıyor.

[continue] ve [not continue] koşullu geçiş

- Bu durum makinesinde temsil edilen nesnenin özellikleri üzerinde mantıksal test ile sağlanır.

Entry ve exit ile belirlenen eylemler, "idle" durumundan gelen ve giden geçişlerde alınacak aksiyonları ifade eder. Hiçbir şey yapılmadan önce, "readCard" yapılacak, "active" durumundan çıkılırken ejectCard yapılacak.



Eşzamanlılık

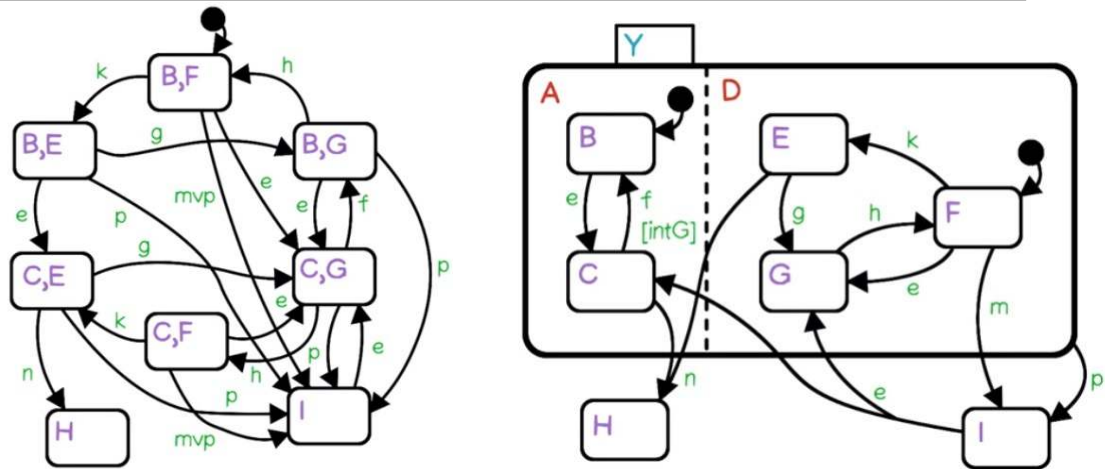
Durumun içinde kesikli çizgi ile ayrılan alanlar ile temsil edilir.

Her makinenin mevcut durumları olabilir, geçişlere tepki verebilirler ve her biri geçişlerde aksiyon gerçekleştirebilirler.

Çarpımlı kombinasyon yerine toplamı kombinasyona olanak verir.

Bir sonraki slaytta Spagetti stilinde eşzamanlılıkta hangi durumdayız takip etmek zordur. Y, H ve I ile eşzamanlı A ve D durumları ile karmaşa azaltılmıştır.

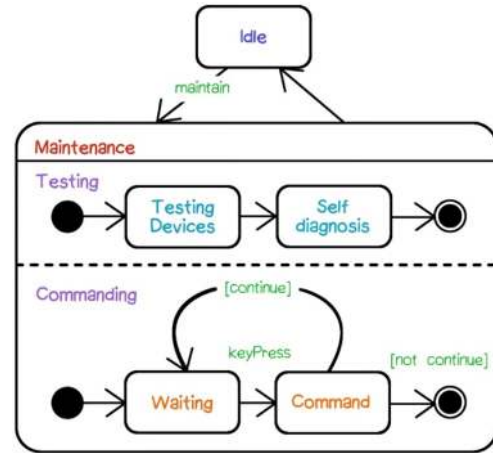
Eşzamanlılık



Eşzamanlılık - UML Gösterimi

"idle" ve "maintenance" durumları.

"maintenance" içinde eş zamanlı iki makine (testing ve commanding)



Senkronizasyon

Eşzamanlı çalışan makineleriniz varsa, bir şekilde işbirliği yapmaları beklenir.

- Yoksa neden diyagrama koyduk ki?

Bu işbirliği için koordinasyon veya senkronizasyon da gereklidir.

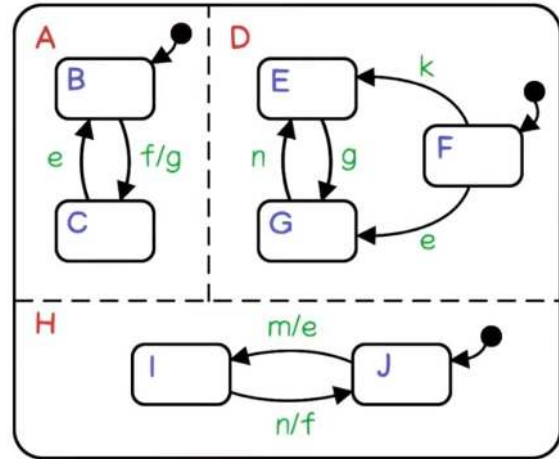
- Yayım [ardarda] Olayları (broadcast [cascade] events)
- Koşullu geçişler (Veri koşulları)

Senkronizasyon- Yayım (ardarda) olaylar

A, D ve H 3 eşzamanlı makine.

Aralarında geçişler var. Bazılarında aksiyonlar (nesneye yönelik sınıf diyagramlarımızda metod çağrıları gibi) var. Fakat, başka bir olayın başlaması da olabilir.

- A'da B'den C'ye F geçişi var. Bu oluyorsa, aynı zamanda G olayı da oluşur diyoruz. Olayları ardarda koyduk.
- Olaylar genel görünür, bu yüzden eşzamanlı makineler de bilir.



Senkronizasyon - Koşullu Geçişler

Köşeli parantezlerle verinin olabileceği durumlara göre koşulları belirtiriz.

- Boolean özellik değerleri
- Değerler sürekli izlenir ve bir değer doğru olduğunda, ilgili geçiş gerçekleşir.

In ve Not In anahtar kelimeleri de desteklenir.

- In: başka bir eşzamanlı makinede x durumunda ise, bu geçişi yaparız.

Koşullu geçiş değişkenleri tüm eşzamanlı makinelere global olarak görünürdür.

Özel Geçişler

UML anahtar kelimelerle belirtilen özel geçişlere olanak verir.

"Active" durumundan "idle" durumuna geçiş etiketi "2 saniye". Sanki bir geri sayaç var ve "active" durumunda 2 saniye kalınca, "idle" duruma geri dönüyor.

"idle" durumunda kendine geçiş var. Tam belirli 11.49 saatinde geçiş yapıyor.

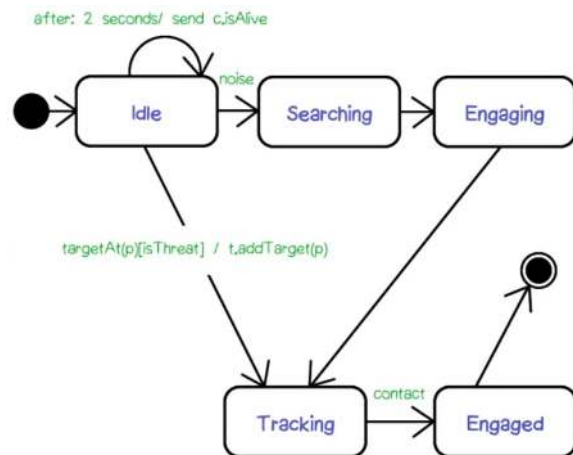


Özel Geçişler

"Idle" durumunda kendine geçiş, diğer durumlarda normal geçişler var.

"idle" ve "tracking" arasındaki geçişte bir eylem var. Bir metodu tetikliyor ve metodun "p" parametresi var.

- Eylem çağrılarında bilgi aktarabiliriz.
- Geçişe neden olan olayda da "p" parametresi vardı. Bilgiyi aktardık.



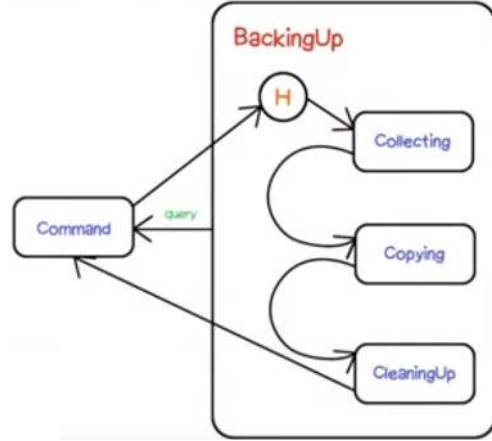
Geçmiş Durumlar

"Command" ve "backing up" harici durumlarımız var.

"backing up"tan "command"a iki geçiş var.

"command"tan çıkan geçiş içiçe durumda H etiketli çembere giriyor.

- Geçmiş (history)
- BackingUp makinesine en son geldiğimizde çıkarken hangi durumda olduğumuzu hatırlar. Son kaldığımız durumdan başlar.
- Geçmiş durum * ile yazılırsa ,iç içe makinelerdeki geçmiş durumlara da dönebiliriz. Güçlüdür ancak, okunurluğu ve takibi çok zorlaştırır.



UML Durum Tanımı

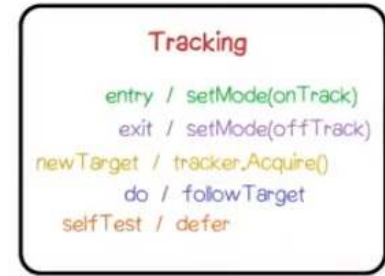
En üstte durumun adı.

Girdi ve çıktıda varsa alınacak aksiyonlar: entry, exit

Girdi-çıkıtlı işlemleri olmadan kendine geçiş: newTarget

Aksiyonlar aç/kapa gibi zaman almayan eylemler, aktiviteler ise zaman alan eylemler: do

Kuyruğa eklenip daha sonra işletilen eylemler: defer



UML Geçiş Tanımı

Geçişin geldiği Kaynak durumu

Geçişin gittiği Hedef durumu

Geçiş tetikleyen olay.

Geçiş üzerinde kontrol (guard).

Geçiş olurken alınabilecek aksiyon

Geçişte Ayrılma ve Birleşmeler olabilir.

Sınıf ve Durum Diyagramı İlişkisi

Geçiş diyagramı oldukça güçlü bir yapıdır.

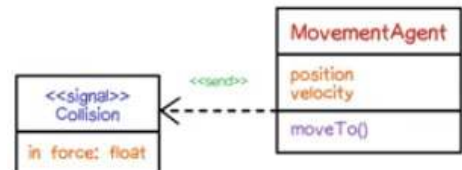
- Tüm özelliklerini her diyagram çiziminizde kullanmanıza gerek yoktur.

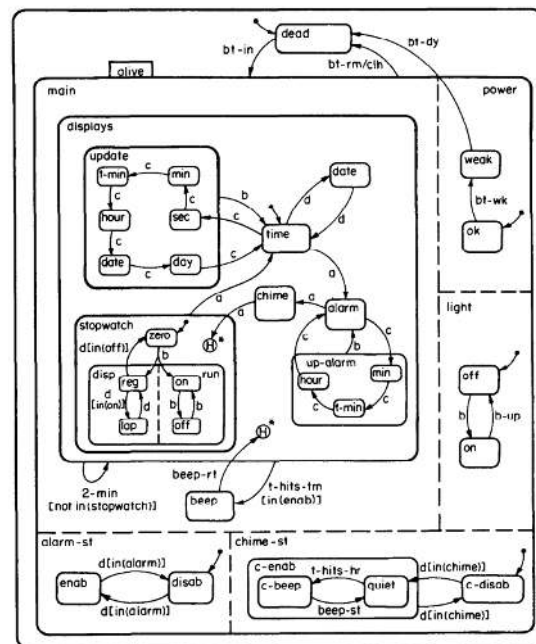
Sınıflarımızın özellikleri bir durum uzayı (state space) yaratır.

Her sınıfın ayrı bir durum diyagramı olabilir.

- Ancak genellikle sınıfların çoğunun basit durumları vardır ve metotlar yeterlidir.
- Tüm sınıflarınız için durum diyagramı çizmenize gerek yoktur.

Durum diyagramındaki özellikler sınıf özelliklerini, aksiyonlar metotları, olaylar da sinyalleri (bağımlılıkları) gösterir.





Özet

Reaktif sistemleri inşa etmek zordur.

Karmaşık sistemlerin kilit durumlarına girdiğinin pek çok örneği ile karşılaşmışsınızdır.

Durum diyagramları bu durumları yönetebilmeniz için bir araçtır.

UML'de davranış modellemesi için aktivite, ardışıl, işbirliği, kullanım senaryosu, haberleşme, zamanlama ve genel etkileşim diyagramlarımız da mevcuttur.

UML dışında zamansal mantık (temporal logic) ve işlem cebiri (process algebra) gibi yöntemler bulunmaktadır.



Radıyo Saat Örneđi

Tanımlar

Radyo elektrikle çalışmaktadır. Pili yoktur.

Bir ses ve bir frekans ayar düğmesi ile kontrol edilmektedir..

- Seçilen frekans, küçük dikey beyaz bir çubuk ile gösterilmektedir.

Ekranda 12 saatlik zaman ve iki küçük lamba vardır.

- Birincisi (üstteki öğleden önce/sonra (AM/PM)
- İkincisi (alttaki) alarm kurulu/değil.



Tanımlar

İki şalter ve 5 düğmesi vardır.

- İlk şalter AM/FM bant seçici
- İkinci şalter 4 pozisyonlu: Soldan sağa: Radyo açık, radyo kapalı, radyo ile alarm, bipleme ile alarm
- Soldaki 4 düğme, saatteki çeşitli zamanlayıcıları ayarlamak için. (saat, dakika, uyanma, uyuma) Uyanma, alarmın ne zaman çalışacağı, uyuma, ne kadar süre sonra otomatik kapanacağı. İkisine de basılmadığında zaman ayarı. Saat ve dakika ile her bir sayaç ayarlanıyor.
- Yarın, şimdiki zamandan 15 dakika sonra uyandırılmak istiyorsanız, uyanma butonuna basılı tutup dakikayı 15 kere basmalısınız.
- 5. düğme, uyuklama düğmesi. Her basışınızda alarm 10 dakika ertelenir.
- Alarm, unutma durumlarına karşı her halde 1 saat sonra otomatik olarak susar.



Durum Diyagramı ile Radyo Saatin Dış Davranış Modellemesi

İlk olarak kullanım senaryolarına bakarız.

- Zamanı ayarlama (alarmı ve uyumayı da ayarlamak için ayrı senaryolar)
- Frekansı değiştirme
- Uyuklamaya basma
- ... Daha pek çok senaryo. Ör. Radyo dinleme

Durum diyagramımız olası tüm senaryoları kapsamalı. Hata durumları da dahil (olmasını istediğiniz şey gerçekleşmediğinde radyo ne yapacak?)

.

Görünen özellikler (Percepts)

Uygulamaya ait özelliklerden, kullanıcının görebildikleri, etkileşebildikleri.

Radyo saattekiler:

- Zaman göstergesi
- Ve iki Işık
- Zamanlayıcı düğmeleri. Ancak bunlar bir duruma sahip değil. Bu yüzden bir cisim olarak değil, bir olay olarak (düğmeye basmak/çekmek) modellemeliyiz.
- Hareket edebilen beyaz çubuk
- Frekans ayar düğmesi
- Ses ayar düğmesi
- AM/FM şalteri
- Mod şalteri
- Uyuklama düğmesi
- Hoparlör

Zaman göstergesinde kaç farklı durum olabilir?

İki ışığı düşünmeyelim.

Saatler 01..12 arası

Dakikalar 00..59 arası

$$12 \times 60 = 720$$

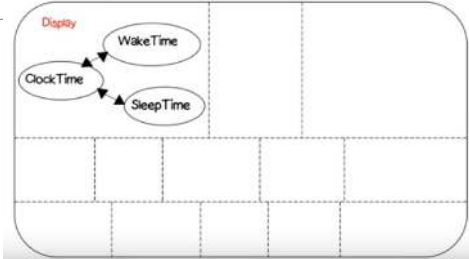
720 durumu olan bir sonlu durum makinesi ile modelleyebiliriz.

Durum diyagramı ile iki eşzamanlı makineyle $12+60 = 72$ duruma anıatabırız.

Bu bile fazla. Saat isimli tek bir düğüme soyutlarız. Beklendiği şekilde 720 durumun oluştuğunu varsayalım.

Gösterge eşzamanlı bir alt makine olarak çalışır. Saati, Uyanmak istediğiniz saati, Kapanma (uyku) saatini gösteren üç durumu vardır.

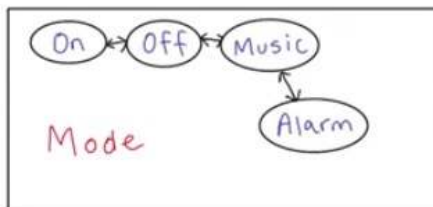
Oklar iki yönlü ama aslında iki ayrı geçiş !



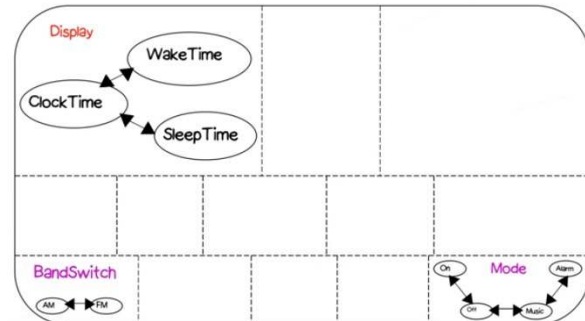
Mod Şalteri Durumları

Kayan "mod şalterinde" cihaz kapalı, radyo, alarm saatinde radyo ve alarm saatinde sadece alarm seçenekleri var.

Bunu FSM ile nasıl modelleriz?



Bunlar da gösterge ile eş zamanlı alt makineler →



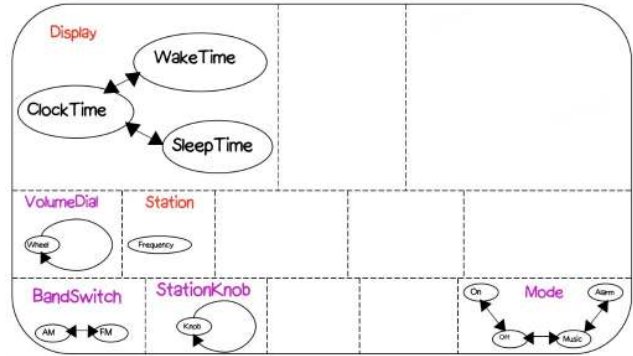
İstasyon Göstergesi

Hareket edebilen beyaz çubuk

Frekans ayar düğmesi ile direk olarak kontrol ediliyor.

Sonsuz sayıda konumda olabilir.

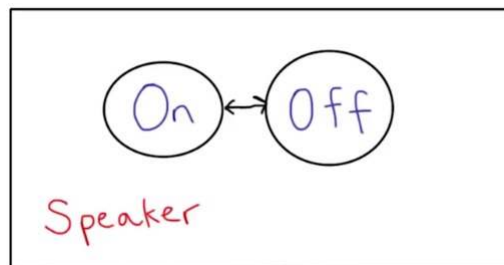
- Geçişsiz tek bir durumda gösterebiliriz.
 - Kullanıcı kendi başına göstergesi değiştiremez
 - Ayar düğmesini çevirir, bu sayede gösterge değişir
- Frekans ayar düğmesi döndükçe istasyon göstergesinin de hareket ettiğini tanımlamalıyız.
 - Biraz ileride.



Hoparlör

Elektronik detaylarına girsek, voltaj seviyeleri vs. pek çok durumumuz olabilir.

Yazılım açısından, sadece hoparlör açık ya da kapalı durumları bizim için yeterli.



Neredeyiz?

Radyo saatin davranışını modellemek için Durum diyagramı geliştiriyoruz.

Radyonun görünen özelliklerine ait davranışlar için şimdiye kadar 7 eşzamanlı makine kullandık.

- Boş yerlere diğer makineleri ekleyeceğiz.

Durum geçişlerini şimdilik gözardı ettik.

- Durum geçişlerine neden olan olaylara göz atalım.

Harici Kontroller ve Tepkiler

Davranış, cihazın bu olaylara nasıl tepki verdiği.

- Her bir harici kontrol, cihaza hangi tepkileri verdiriyor/olayı sağlıyor?

Mod şalteri: sola kaydır, sağa kaydır.

- İki bağımsız olay olabilir, ya da bir parametresi olan kaydır olayı olabilir.

Frekans ayar düğmesi: döndür (parametresi açı)

AM/FM şalteri: sola kaydır, sağa kaydır.

Uyuklama düğmesi: bas

- Kullanıcı açısından olaylara bakıyoruz. Alarm çalmazken de basabilir. Bu durumda ne olacağını bize durum diyagramımız söyleyecek.

Uyanma düğmesi: bas, çek

- Saat ve dakika düğmeleri: bas



Eylemlerden Olaylara

Kullanıcı aksiyonlarını olaylara (tepki vermemiz gereken anlık oluşum) çeviririz.

Tüm olayları ve varsa bunların kombinasyonlarını ele almalıyız.

1. Ses ayar düğmesini çevir
2. AM bandını seç
3. FM bandını seç
4. İstasyon seç
5. Fişten çıkar
6. Fişe tak
7. Uyku saati düğmesine bas
8. Uyku saati düğmesini bırak

Analizde hepsini listelemek ve parametrelerini belirlemek gerekir.

Sistemin bunlara ne tepki vereceği belirlenir.

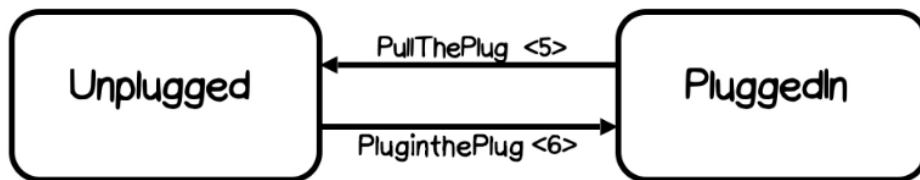
9. Uyanma saati düğmesine bas
10. Uyanma saati düğmesini bırak
11. Mod seçim düğmesini sağa kaydır
12. Mod seçim düğmesini sola kaydır
13. Uyuklama düğmesine bas
14. Saat düğmesine bas
15. Saat düğmesini bırak
16. Dakika düğmesine bas
17. Dakika düğmesini bırak

En Dış Durumlarımız nelerdir?

Fişe takılı olabiliriz yada fişimiz çekilmiş olabilir.

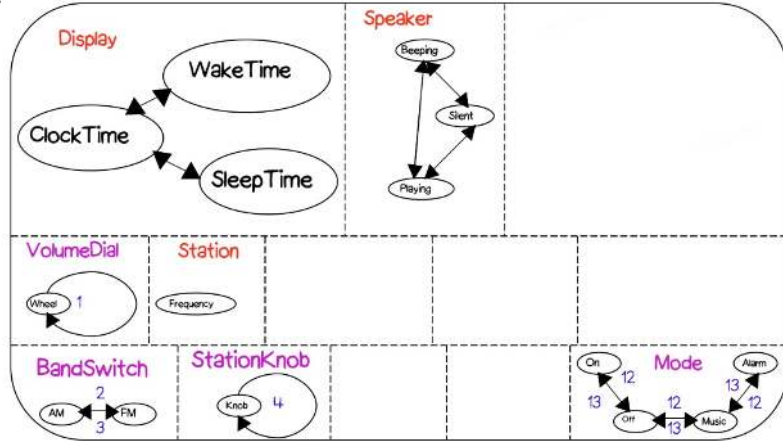
Bu iki durum arasında 5. ve 6. olaylarla geçiş yaparız.

Bundan sonrasında, hep fişe takılı durumunun içini modelleyeceğiz.



Olayların Durum makinesine yerleşimi

1. Ses ayar düğmesini çev
2. AM bandını seç
3. FM bandını seç
4. İstasyon seç
12. Mod seçim düğmesini sağa kaydır
13. Mod seçim düğmesini sola kaydır



Yeni alt makineler

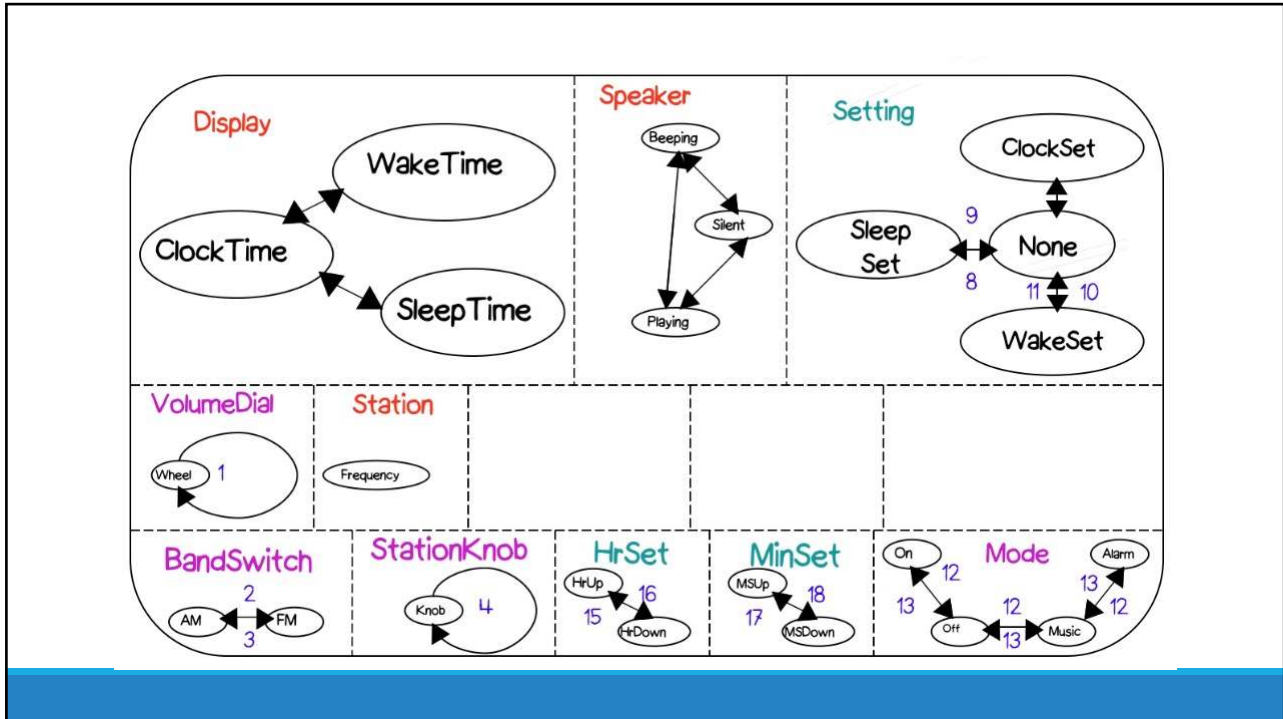
Olaylarımız, yeni eşzamanlı alt makinelere ihtiyaç duyduğumuzu gösteriyor.

Örneğin uyuklama düğmesine basınca, 10 dakika sonra tekrar alarm devreye girmeli.

- Kullanıcıya görünür değil ama kullanıcıyı etkileyen bir iç sayaç var gibi.
- Bunları da modellemeliyiz.

Uyku ve uyanma düğmelerine bastığımızda, bir duruma gireriz.

- Bu durumda saat ve dakika düğmelerine basarsak da ayar durumuna gireriz.



Olaylara verilen Tepkiler

Radyomuzun kullanıcıya sunduğu gerçek değeri, bu olaylara verdiği tepkilerdir.

Listelediğimiz tüm olayların radyonun durumunda bir etkisi olması gerekir.

Ör. Ses ayar düğmesini çevirdiğimizde, hoparlörden gelen sesin artmasını bekleriz.

- Ayrıca, düğmenin açılma konumu da değişir.
- Bunu da tepki olarak göstermeliyiz. (Gece karanlıkta sesi ayarlıyorsa?)

#	Uyarı	Tepki
11	Mod seçim düğmesini sağa kaydır	Açık → Kapalı Kapalı → Müzik Müzik → Alarm
12	Mod seçim düğmesini sola kaydır	Alarm → Müzik Müzik → Kapalı Kapalı → Açık

Uyaran Tepki Tablosu

#	Uyarı	Tepki
1	Ses ayar düğmesini çevir	Düğmenin pozisyonunu değiştir Hoparlörün sesini arttır
2	AM bandını seç	Düğmenin konumunu değiştir Bandı değiştir
3	FM bandını seç	Düğmenin konumunu değiştir Bandı değiştir
4	İstasyon seç	Düğmenin pozisyonunu değiştir Çalan istasyonu değiştir İstasyon göstergesini değiştir
5	Fişten çıkar	Fişten çıkarılmış durumuna geç
6	Fişe tak	Fişe takılı durumuna geç, tüm eşzamanlı makinelerin varsayılan durumlarına geç

Zamanlı olaylar

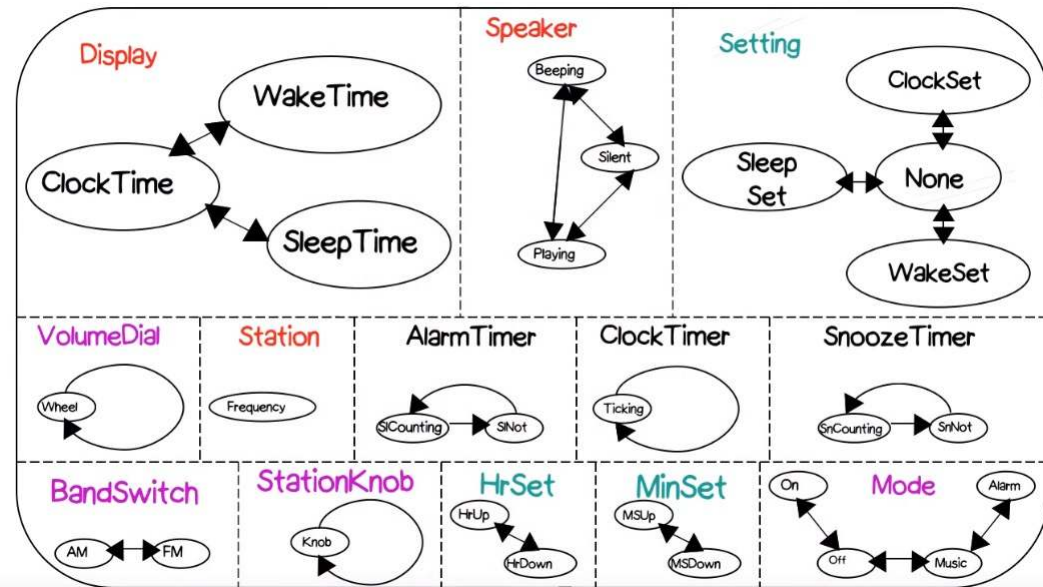
Radyonun saati, alarmin ayarlandığı saate geldiğinde ne tepki verilmesi gerekir?

1. Bip/radyo çal
2. Alarm susma zamanlayıcısını başlat
3. Hoparlörü aç

İç Durumlar

Her ne kadar dış uyaranlara karşı durumları modellesek de, sistemin tam olarak çalışması için kullanıcılara görünmeyen (zamanlayıcılar gibi) iç durumlara da ihtiyaç duyarız ve bunları da modellememiz gerekir.

- Uyuklama → başla, 10 dakikaya kadar say
- Alarm → başla, 60 dakikaya kadar say
- Uyku → başla, ayarlanan zamana kadar (max 60 dakika)
- Saat zamanı → her 1 dakikada değiştir.



Diğer İç Olaylar

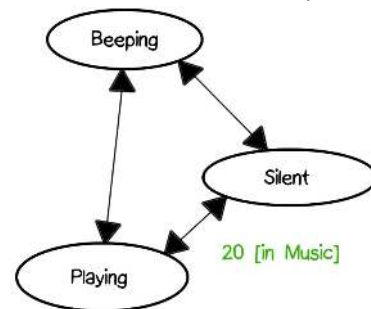
#	Uyarı	Tepki
19	Alarm sayacı bitti	If not in Hoparlör:Sessiz go to Hoparlör:Sessiz
20	Zaman, alarm saatine geldi	If in Mod:Muzik go to Hoparlör:Çalıyor If in Mod:alarm go to Hoparlör:Bipliyor
21	Uyuklama sayacı bitti	If in Mod:Muzik go to Hoparlör:Çalıyor If in Mod:alarm go to Hoparlör:Bipliyor
22	Zaman, uyanma saati +1 saate geldi	If in Hoparlör:Çalıyor go to Hoparlör:Sessiz If in Hoparlör:Bipliyor go to Hoparlör:Sessiz

Kontrollü (Guarded) Geçişler

Alt makinenin belirli bir durumda olması ile kontrol edilecek şekilde bir olaya tepki verilen durumlar

If in Mod:Muzik go to Hoparlör:Çalıyor

- Sessiz durumu ile Çalıyor durumu arasındaki geçişin kontrolü, Müzik durumunda olmaya bağlı
- Köşeli parantez ile gösteririz.



Yayım/Ardarda (Cascaded) Olaylar

Aktiviteleri koordine etmenin bir diğer yolu.

Bir iç olay yayımlanır. Tüm durumlar tüm olayları dinlediği için, eşzamanlı çalışan alt makinelerde senkronizasyon için kullanılabilir.

Frekans ayar düğmesi çevrildiğinde, üç tepki gerekli:

- Düğme fiziksel olarak yeni pozisyona gelmeli
- Radyo kanalı değişmeli
- O anki istasyonu gösteren beyaz çubuk hareket ettirilmeli → bu ayrı bir eş zamanlı makinede! Haberdar edilmeli.
 - Yeni bir iç olay yaratırız. İstasyon makinesi buna tepki verir.

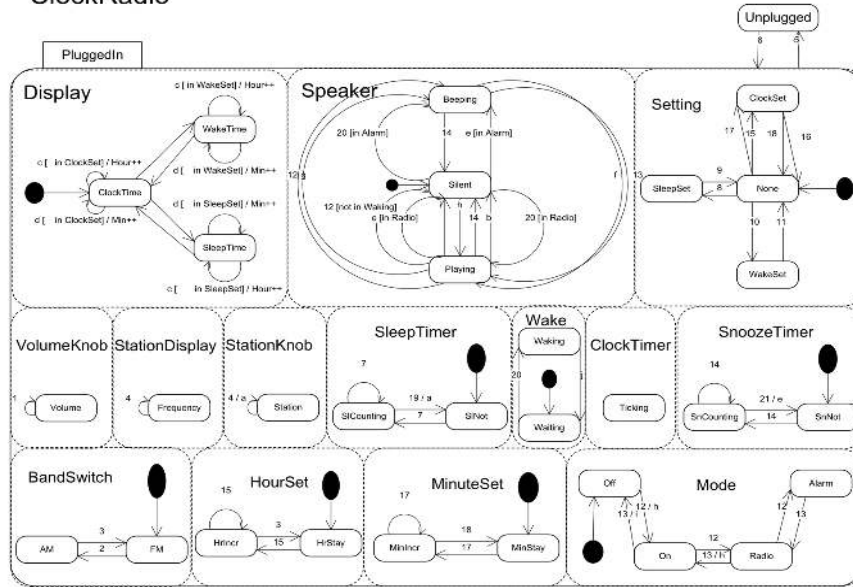
Yapabileceklerimiz

Bu şekilde yeni olaylar ekleyerek, sistemimizin istediğimiz gibi davranmasını sağlamayı garanti edebiliriz.

Tam bir model için şunları da tamamlamamız gerekli:

- Her eşzamanlı makinenin varsayılan durumları belirlemeliyiz.
- Geçişlere gerekli olan kontrolleri [guard] yerleştirmeliyiz.
- Geçişleri sağlamak için gerekli olan iç olayları yaratmalıyız.

ClockRadio



Geçerleme

Durum diyagramımızı tamamladıktan sonra geçerleme/validasyonunu da yapmamız gerekir.

Bir ekip ile gözden geçirme yapabiliriz. Tüm kullanım senaryolarının üzerinden geçerek, sistemin tam olarak beklediklerinizi yaptığını doğrulayabiliriz.

Model kontrolü yapabiliriz. Diyagramı gerçekleyip üzerinde testler koşturabiliriz ve beklediğimiz şekilde olayların ve geçişlerin oluşup oluşmadığını test edebiliriz.

Bir simülatör yaratıp sistemin çalışmasını simüle edebiliriz.

Yeni sorular oluşursa yine kullanıcıya döneriz.

Durum Diyagramı Modelleme Yöntemi

1. Kullanım senaryolarını hazırla
2. Görünen (harici) özellikleri (percepts) belirle
3. Görünen özellikleri durumlarla (eşzamanlı makinelerle) modelle
4. Harici kontrolleri ve uyarıları belirle
5. İlave durumlar ve geçişler/olaylar ile modelle
6. Uyarı/tepki ikililerini listele
7. Gerekli (ör. Zamanlayıcılar) iç durumları belirle
8. koordinasyon mekanizmalarını oluştur
9. Eylemleri/aktiviteleri ekle
10. Senaryolar ile sonuç makinesini geçlerle



Yazılım Mimarisi
